Session 01: Introduction to AI and Python Environment Setup

This session introduces the foundational concepts of Artificial Intelligence and equips learners with the essential tools required for hands-on work throughout the course. The focus is on understanding what AI is, preparing a productive development environment, and completing initial practical exercises using core scientific Python libraries.

1. What Is Artificial Intelligence?

An overview of AI as a field focused on building systems that can perform tasks typically requiring human intelligence. Core ideas include learning from data, recognizing patterns, making decisions, and automating reasoning.

2. Tools and Technologies

Below is a concise guide for each essential tool used throughout the course.

Git and GitHub

Version control is critical for tracking changes and collaborating effectively.

Install Git:

- Install from https://git-scm.com/install
- Verify installation:

```
git --version
```

Set up Git identity:

```
git config --global user.name "Your Name"
  git config --global user.email "your@email.com"
```

GitHub usage overview:

- Create a repository on GitHub.
- Clone it locally:

```
git clone <repository-url>
```

- Commit and push changes:

```
git add .
  git commit -m "Initial commit"
  git push origin main
```

Visual Studio Code

A lightweight and extensible code editor.

- Download from https://code.visualstudio.com
- Recommended extensions: Python, GitHub, Jupyter, Pylance.

Python

Python is the primary programming language for AI and data science.

- Download from https://www.python.org/downloads/
- Verify installation:

```
python3 --version
```

A walkthrough of the essential tooling that supports modern AI workflows:

- Git and GitHub for version control and collaboration.
- Visual Studio Code for an extensible and efficient development workflow.
- Python as the primary programming language for applied AI.
- Jupyter Notebook for interactive and exploratory programming. Jupyter is

  especially important in AI work because it enables rapid experimentation, step-by-step execution, inline visualizations, and clear documentation of data workflows. It supports an iterative, research-oriented workflow ideal for data exploration and model development.

3. Virtual Environments vs Global Python

Using a global Python installation often leads to dependency conflicts, version mismatches, and inconsistent environments across machines. Global installations mix libraries from different projects, making systems fragile and difficult to maintain.

Virtual environments (venvs) provide isolated, project-specific environments that contain only the dependencies required for that project.

Why virtual environments are better:

- Prevent dependency conflicts by isolating packages per project.
- Ensure reproducibility across machines and team members.
- Protect system-level Python from accidental modification.
- Make development environments portable and easier to debug.

This course relies on virtual environments to guarantee consistent behavior in all notebooks, scripts, and exercises.

4. Setting Up the Python Environment with Poetry

Poetry manages dependencies, virtual environments, and project configuration.

Why Install Poetry with pipx?

pip installs Python packages into a single environment — either your system Python or an active virtual environment. When you run `pip install poetry`, Poetry and all of its dependencies are added to that same environment alongside every other package already installed there. Over time, upgrading one tool can break another because they share the same dependency pool.

pipx is built specifically for installing Python **applications** (command-line tools such as Poetry, Black, or HTTPie). Each application is installed into its own isolated virtual environment. pipx then exposes the app's command on your `PATH`, so you can run `poetry` as usual — but its dependencies never mix with your system Python, your project venvs, or other CLI tools.

	`pip install <tool>`	`pipx install <tool>`
What it installs	Libraries and apps into one environment	CLI applications only, one per environment
Isolation	Shared with everything else in that environment	Each app gets its own venv
Best for	Project dependencies (`numpy`, `pandas`, ...)	Standalone tools (`poetry`, `black`, ...)
Risk	Dependency conflicts between tools	Minimal — tools cannot interfere with each other

This course uses **pipx** to install Poetry because Poetry is a project-management tool, not a library your code imports. Keeping it isolated prevents version conflicts and makes upgrades safe.

Install pipx and Poetry

Windows

Install Scoop (a command-line package manager for Windows):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
  Invoke-RestMethod -Uri https://get.scoop.sh | Invoke-Expression
```

Install pipx:

```
scoop install pipx
  pipx ensurepath
```

Restart your terminal after `pipx ensurepath` so the updated `PATH` takes effect.

Install Poetry:

```
pipx install poetry
```

macOS / Linux

Install Homebrew (if not already installed):

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/inst
  all.sh)"
```

Install pipx:

```
brew install pipx
  pipx ensurepath
```

Restart your terminal after `pipx ensurepath` so the updated `PATH` takes effect.

Install Poetry:

```
pipx install poetry
```

Verify installation

```
poetry --version
```

Initialize a Poetry Project

```
poetry init
```

Follow the interactive prompts to define project metadata.

Create and Activate a Virtual Environment

Poetry automatically manages the virtual environment.

Activate the environment:

```
poetry env activate
```

To use a spicific Python version:

```
poetry env activate
```

Check environments:

```
poetry env list
```

Install Project Dependencies

Poetry installs everything listed in `pyproject.toml`.

```
poetry install
```

To add a package:

```
poetry add numpy pandas matplotlib
```

A structured guide to creating a clean, isolated development environment using Poetry:

- Installing pipx and Poetry (each tool in its own isolated environment).
- Creating a project environment.
- Managing dependencies.
- Running scripts inside the environment.

- Best practices for reproducibility and project organization.

5. Practical Work: Core Scientific Libraries

Hands-on exploration using:

- NumPy for numerical computing.
- Pandas for data manipulation.
- Matplotlib for visualizations.

Learners should complete small exercises demonstrating:

- Array manipulation.
- Loading and cleaning datasets.
- Producing informative charts.

Challenge Exercise

Create a Python script or notebook that:

1. Loads a small dataset using Pandas.
2. Performs at least three transformations (filtering, grouping, or deriving new columns).
3. Visualizes one meaningful insight using Matplotlib.
4. Runs entirely inside the Poetry-managed virtual environment.